

**LAPORAN PRAKTIKUM ALGORITMA DAN
PEMROGRAMAN 2**

WEEK 12

“Searching”



Disusun Oleh :

Farel Tri Julian
109082500163

Dosen :

WAHYU ANDI SAPUTRA, S.Pd., M.Eng

**PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2025/2026**

Dasar teori

Pencarian merupakan salah satu algoritma paling mendasar dalam ilmu komputer dan pemrograman yang digunakan untuk menemukan posisi atau keberadaan suatu data di dalam sekumpulan data atau kontainer, seperti array atau list. Berdasarkan kondisi data yang dihadapi, terdapat berbagai macam metode pencarian, salah satunya adalah Pencarian Berurutan atau Sequential Search.

Algoritma Sequential Search bekerja dengan cara memeriksa setiap elemen di dalam kumpulan data satu per satu, dimulai dari elemen pertama hingga elemen terakhir atau sampai data yang dicari berhasil ditemukan. Metode ini sangat fleksibel karena dapat diterapkan baik pada sekumpulan data yang sudah terurut maupun data yang masih acak. Kelemahan utama dari metode ini terletak pada efisiensinya; jika data yang dicari berada di posisi paling akhir atau bahkan tidak ada di dalam kumpulan data tersebut, algoritma harus melakukan iterasi sebanyak n kali. Oleh karena itu, dalam analisis kompleksitas algoritma menggunakan Notasi Big-O, Sequential Search memiliki kompleksitas waktu kasus.

Di dalam bahasa pemrograman Go, implementasi pencarian nilai spesifik sering kali dipadukan dengan struktur perulangan for untuk iterasi data, serta struktur kondisi if-else untuk memvalidasi apakah elemen data yang sedang diperiksa sama dengan nilai target. Apabila nilai target ditemukan, fungsi pencarian umumnya akan langsung mengembalikan nilai indeks posisi elemen tersebut berada. Sebaliknya, jika proses penelusuran telah mencapai batas akhir elemen dan nilai target tetap tidak ditemukan, fungsi biasanya mengembalikan nilai penanda khusus seperti -1 untuk merepresentasikan bahwa data tersebut tidak eksis di dalam sistem penyimpanan.

WEEK 12

Tugas 1

```
package main

import "fmt"

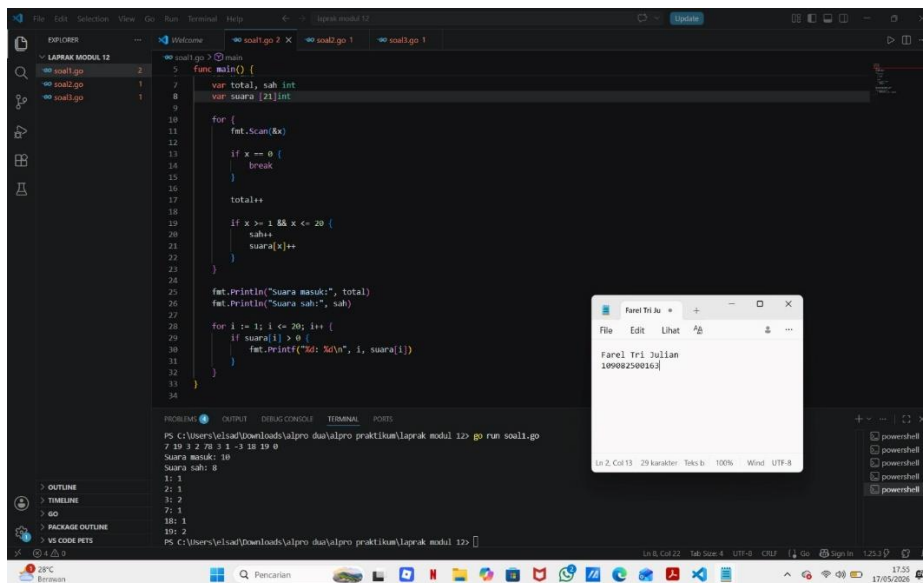
func main() {
    var x int
    var total, sah int
    var suara [21]int

    for {
        fmt.Scan(&x)
        if x == 0 {
            break
        }
        total++
        if x >= 1 && x <= 20 {
            sah++
            suara[x]++
        }
    }

    fmt.Println("Suara masuk:", total)
    fmt.Println("Suara sah:", sah)

    for i := 1; i <= 20; i++ {
        if suara[i] > 0 {
            fmt.Printf("%d: %d\n", i, suara[i])
        }
    }
}
```

Screenshots Output



Deskripsi :

Program tersebut berfungsi untuk menghitung dan mentabulasi perolehan suara pemungutan suara secara real-time, di mana pengguna dapat memasukkan nomor kandidat secara terus-menerus hingga proses dihentikan dengan memasukkan angka 0. Selama proses input, program secara otomatis menghitung total suara yang masuk serta memisahkan suara yang sah ke dalam sebuah array penyimpanan, lalu di akhir program akan menampilkan ringkasan jumlah suara masuk, total suara sah, serta rincian perolehan suara untuk masing-masing kandidat yang berhasil mendapatkan minimal satu suara.

Tugas 2

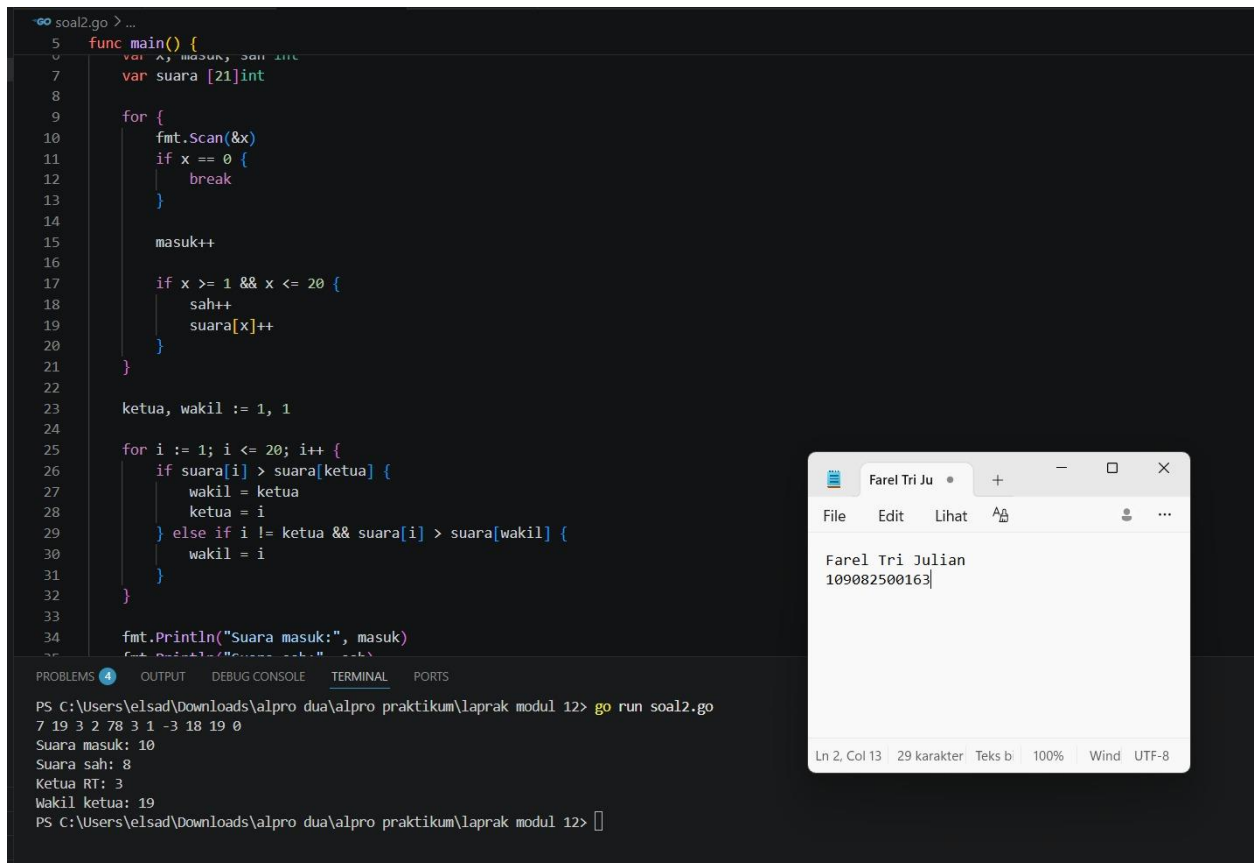
```
package main

import "fmt"

func main() {
    var x, masuk, sah int
    var suara [21]int

    for {
        fmt.Scan(&x)
        if x == 0 {
            break
        }
        masuk++
        if x >= 1 && x <= 20 {
            sah++
            suara[x]++
        }
    }
    ketua, wakil := 1, 1
    for i := 1; i <= 20; i++ {
        if suara[i] > suara[ketua] {
            wakil = ketua
            ketua = i
        } else if i != ketua && suara[i] > suara[wakil] {
            wakil = i
        }
    }
    fmt.Println("Suara masuk:", masuk)
    fmt.Println("Suara sah:", sah)
    fmt.Println("Ketua RT:", ketua)
    fmt.Println("Wakil ketua:", wakil)
}
```

Screenshots Output



The screenshot displays a Go program in an IDE and its execution output in a terminal. The program, named `soal2.go`, implements a simulation for electing a Ketua (Chairman) and Wakil Ketua (Vice Chairman) from 20 candidates based on the number of votes they receive. The code uses a loop to read input votes, validates them, and then iterates through the candidates to find the top two vote-getters.

```
5 func main() {
6     var A, masuk, sah int
7     var suara [21]int
8
9     for {
10        fmt.Scan(&x)
11        if x == 0 {
12            break
13        }
14
15        masuk++
16
17        if x >= 1 && x <= 20 {
18            sah++
19            suara[x]++
20        }
21    }
22
23    ketua, wakil := 1, 1
24
25    for i := 1; i <= 20; i++ {
26        if suara[i] > suara[ketua] {
27            wakil = ketua
28            ketua = i
29        } else if i != ketua && suara[i] > suara[wakil] {
30            wakil = i
31        }
32    }
33
34    fmt.Println("Suara masuk:", masuk)
35    fmt.Println("Suara sah:", sah)
36    fmt.Println("Ketua RT:", ketua)
37    fmt.Println("Wakil ketua RT:", wakil)
38}
```

The terminal output shows the execution of the program with the following results:

```
PS C:\Users\elsad\Downloads\alpro dua\alpro praktikum\laprak modul 12> go run soal2.go
7 19 3 2 78 3 1 -3 18 19 0
Suara masuk: 10
Suara sah: 8
Ketua RT: 3
Wakil ketua: 19
PS C:\Users\elsad\Downloads\alpro dua\alpro praktikum\laprak modul 12>
```

Deskripsi :

Program berfungsi untuk mensimulasikan pemilihan Ketua dan Wakil Ketua RT berdasarkan total perolehan suara terbanyak dari 20 kandidat yang tersedia. Program akan terus menerima input nomor kandidat dari pengguna dan menyimpannya ke dalam array hingga angka 0 dimasukkan sebagai tanda pemungutan suara selesai. Selain menghitung total suara masuk dan memvalidasi jumlah suara sah, program ini mengimplementasikan logika pencarian nilai terbesar pertama dan kedua untuk menentukan pemenang: kandidat dengan suara terbanyak otomatis terpilih sebagai Ketua RT, sedangkan kandidat dengan suara terbanyak kedua ditetapkan sebagai Wakil Ketua RT.

Tugas 3

```
package main

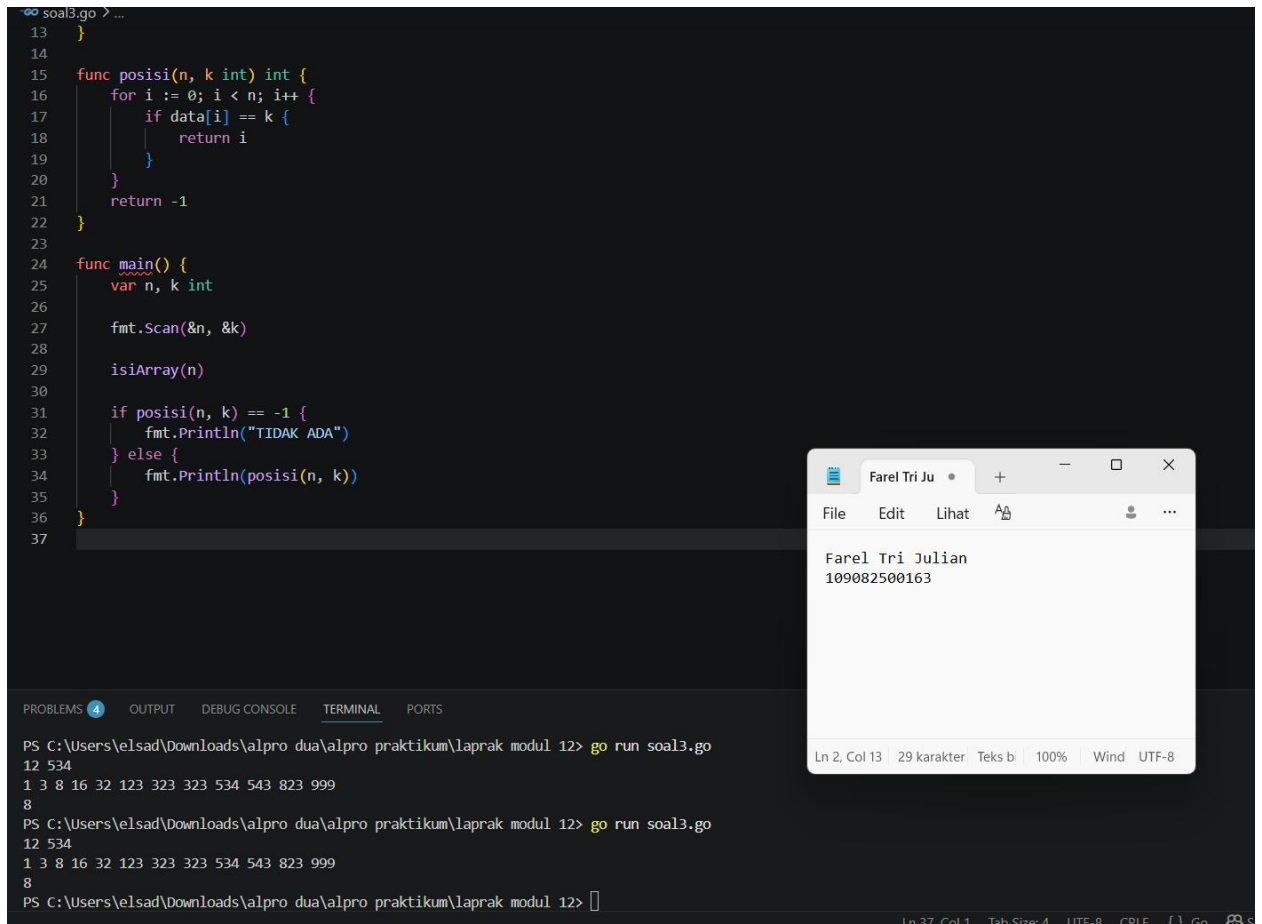
import "fmt"

const NMAX = 100000
var data [NMAX]int
func isiArray(n int) {
    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }
}
func posisi(n, k int) int {
    for i := 0; i < n; i++ {
        if data[i] == k {
            return i
        }
    }
    return -1
}

func main() {
    var n, k int

    fmt.Scan(&n, &k)
    isiArray(n)
    if posisi(n, k) == -1 {
        fmt.Println("TIDAK ADA")
    } else {
        fmt.Println(posisi(n, k))
    }
}
```

Screenshots Output



```
13 }
14
15 func posisi(n, k int) int {
16     for i := 0; i < n; i++ {
17         if data[i] == k {
18             return i
19         }
20     }
21     return -1
22 }
23
24 func main() {
25     var n, k int
26
27     fmt.Scan(&n, &k)
28
29     isiArray(n)
30
31     if posisi(n, k) == -1 {
32         fmt.Println("TIDAK ADA")
33     } else {
34         fmt.Println(posisi(n, k))
35     }
36 }
37
```

```
PS C:\Users\elsad\Downloads\alpro dua\alpro praktikum\laprak modul 12> go run soal3.go
12 534
1 3 8 16 32 123 323 323 534 543 823 999
8
PS C:\Users\elsad\Downloads\alpro dua\alpro praktikum\laprak modul 12> go run soal3.go
12 534
1 3 8 16 32 123 323 323 534 543 823 999
8
PS C:\Users\elsad\Downloads\alpro dua\alpro praktikum\laprak modul 12>
```

Deskripsi :

Program mengimplementasikan algoritma pencarian berurutan untuk menemukan posisi indeks dari suatu nilai target k di dalam sebuah array global data berkapasitas maksimal 100.000 elemen. Program mengawali proses dengan membaca jumlah elemen n dan nilai target k yang dicari, dilanjutkan dengan memanggil fungsi `isiArray` untuk mengisi array sebanyak n bilangan bulat dari input pengguna. Melalui fungsi `posisi`, program melakukan perulangan dari indeks pertama hingga $n-1$ untuk memeriksa keberadaan nilai k ; jika ditemukan, fungsi akan langsung mengembalikan indeks tempat nilai tersebut berada, namun jika perulangan selesai dan nilai k tidak ditemukan, program akan mengembalikan nilai -1 yang kemudian memicu percabangan di fungsi `main` untuk mencetak teks "TIDAK ADA".

Daftar pustaka

- Enterprise, J. (2019). *Pengenalan Pemrograman Pemula menggunakan Golang*. Jakarta: PT Elex Media Komputindo.
- Gadc, A. G., & Kristoko, D. H. (2023). *Algoritma dan Struktur Data dengan Go*. Yogyakarta: Andi Offset.
- Munir, R. (2016). *Algoritma dan Pemrograman dalam Bahasa Pascal, C, dan C++* (Edisi Revisi). Bandung: Informatika Bandung.